

RippleLab: 30-Day MVP Delivery Plan

Project: RippleLab - "What happens to me if the economy changes?"

Build window: 30 focused build days

Initial market: India

Delivery model: One tested, reviewable increment per day, followed by a GitHub commit/push and a typed PDF progress report

1. Outcome at Day 30

The first public MVP will let a user:

- Create a private financial profile covering age, city, household, income, expenses, savings, investments, loans, housing and goals.
- Enter a natural-language what-if question.
- Run five supported scenario families: inflation, repo-rate change, oil-price change, income-tax change, and job loss/salary change.
- Inspect a causal graph showing the chain from macro event to personal impact.
- Compare multiple scenario settings and representative personas.
- View ranges and probability distributions instead of a single false-precision forecast.
- Inspect formulas, assumptions, sources, dates and confidence for every material result.
- Export a shareable report while keeping personally identifying profile data private by default.

The MVP is an educational decision-support tool, not investment, tax or lending advice. Monetary outputs will be estimates under explicit assumptions.

2. Product Rules That Protect the Build

1. **Calculations are deterministic.** The LLM may parse questions and explain results, but it never invents the numeric outcome.
2. **Only five scenario families ship.** Unsupported questions are mapped to the closest supported template or clearly declined.
3. **Every number is traceable.** Each result links to its formula, inputs, units, evidence and confidence.
4. **Ranges beat fake certainty.** Base, optimistic and adverse assumptions are visible; Monte Carlo is used only where uncertainty is meaningful.
5. **Privacy is the default.** Reports omit or mask sensitive profile fields unless the user explicitly includes them.
6. **Mobile and desktop both matter.** Core flows must remain usable at 360 px and above.
7. **A thin vertical slice comes early.** A complete repo-rate journey is finished before expanding to other scenarios.

3. MVP Boundary

Included

- Email/social authentication through Supabase Auth.
- One active personal financial profile per account, with an extensible schema.
- Structured scenario specification validated at the API boundary.
- Deterministic Python calculation engine with versioned formulas.
- Causal graph, personal impact cards, winners-and-losers comparison and scenario comparison.
- Monte Carlo distributions for supported uncertain inputs.
- Source registry using official/primary sources where available.
- Saved scenarios, share links with redacted data and PDF export.
- Automated tests, CI, observability, accessibility checks and deployment.

Deferred until after the MVP

- Bank-account aggregation, transactions or credit-bureau integration.
- Real-time trading or investment recommendations.
- State-by-state tax edge cases and full professional tax planning.
- Arbitrary user-authored economic models.
- Native mobile apps, multi-currency support and non-India policy engines.
- Collaborative editing, organizations and paid subscriptions.

4. Target Architecture

```

Next.js web app
|-- Supabase Auth + PostgreSQL
|-- React Flow causal graph
|-- Recharts distributions/comparisons
|-- Typed API client
|
FastAPI economic service
|-- Scenario schema and validation
|-- Deterministic calculators
|-- Monte Carlo runner
|-- Evidence/source registry
|-- Versioned result contract
|
LLM boundary
|-- Natural language -> constrained scenario JSON
|-- Result JSON + evidence -> plain-language explanation
|-- Never performs authoritative calculations
|
Deployment
|-- Vercel: web
|-- Render/Railway: API
|-- Supabase: auth/database

```

Recommended monorepo layout:

apps/web	Next.js application
services/economic-engine	FastAPI service
packages/contracts	Shared JSON Schema / generated TypeScript types
packages/config	Shared lint and TypeScript configuration
data/sources	Versioned evidence metadata and snapshots
docs	Architecture, formulas, decisions and reports
tests/e2e	Browser-level critical journeys

5. Definition of Done for Every Day

A day is complete only when all applicable items pass:

- The day's acceptance criteria work locally.
- Unit, integration and/or end-to-end tests cover the new behavior.
- Lint, type-check and build pass for affected applications.
- No secrets, personal test data or generated temporary artifacts are committed.
- Documentation and the changelog are updated.
- Changes are committed with a focused Conventional Commit message and pushed to the active GitHub branch.
- A typed PDF daily report is generated with completed work, proof, changed files, decisions, risks and next-day scope.

Suggested commit format: `type(scope): concise outcome`, for example `feat(engine): add repo-rate impact calculator`.

6. Thirty-Day Workflow

Week 1 - Foundation and the first end-to-end slice

Day 1 - Product contract and repository bootstrap

Build: Initialize the monorepo, Next.js web app, FastAPI service, shared contracts package, formatting/linting, environment examples and base README. Record MVP boundaries, user journeys and non-advice disclaimer. Connect the GitHub remote and protect secrets.

Acceptance: Web and API start locally; one command runs core checks; repository contains no real credentials; CI skeleton is present.

Git checkpoint: `chore(repo): bootstrap RippleLab monorepo`

Day 2 - Design system and application shell

Build: Create color, typography, spacing and data-visualization tokens; responsive navigation; dashboard shell; empty/loading/error states; reusable buttons, cards, fields, dialogs and confidence badges.

Acceptance: Component showcase renders on mobile and desktop; keyboard focus is visible; automated accessibility smoke check has no critical violations.

Git checkpoint: `feat(ui): establish RippleLab design system and shell`

Day 3 - Authentication and protected routing

Build: Configure Supabase, sign-up/sign-in/sign-out, session handling, protected dashboard routes, recovery flow and local development setup.

Acceptance: Anonymous users cannot open protected routes; a test user can complete the auth lifecycle; auth failures are understandable.

Git checkpoint: `feat(auth): add Supabase authentication flow`

Day 4 - Economic digital-twin profile

Build: Model profile fields and migrations; create a multi-step form for demographics, income, expenses, savings, investments, loans, housing and goals; add validation and consent copy.

Acceptance: A user can create, edit and reload a profile; rupee values and percentages validate correctly; database row-level security isolates users.

Git checkpoint: `feat(profile): build personal financial profile`

Day 5 - Scenario and evidence contracts

Build: Define versioned request/result schemas, scenario enums, causal-node/edge model, assumption model, citation metadata and confidence rubric. Generate TypeScript types from the canonical schema.

Acceptance: Invalid units and unsupported scenario types fail clearly; web and API contract tests agree; a golden example is documented.

Git checkpoint: `feat(contracts): define versioned simulation schema`

Day 6 - Loan and deposit calculation primitives

Build: Implement EMI, outstanding-balance, floating-rate reset, interest-income and rate-pass-through functions using Decimal-safe money handling. Document formulas and rounding.

Acceptance: Unit tests cover known examples, boundaries, zero/negative validation and partial pass-through; every output exposes formula version and assumptions.

Git checkpoint: `feat(engine): add loan and deposit primitives`

Day 7 - Repo-rate vertical slice

Build: Connect profile -> repo-rate scenario form -> FastAPI calculation -> impact summary. Calculate EMI and deposit-income changes with configurable transmission assumptions.

Acceptance: A seeded homeowner/saver profile produces reproducible before/after results; the UI never asks the LLM to calculate money.

Git checkpoint: `feat(repo-rate): deliver first end-to-end simulation`

Week 1 gate: A signed-in user can save a profile and run one trustworthy scenario end to end.

Week 2 - Core economic engines and causal visualization

Day 8 - Causal graph framework

Build: Render the repo-rate causal graph with React Flow; add node categories, directional edges, zoom/fit controls and a side panel containing mechanism, formula, assumption, source and confidence.

Acceptance: Selecting any material node or edge reveals its explanation; graph remains usable on small screens and with keyboard navigation.

Git checkpoint: `feat(graph): add inspectable causal graph`

Day 9 - Inflation engine

Build: Add category-weighted household inflation, real purchasing-power change, salary adjustment and real return calculations. Separate headline assumptions from a user's personal expense basket.

Acceptance: Users see which spending categories drive their result; tests cover high/low inflation and salary growth combinations.

Git checkpoint: `feat(inflation): add household impact model`

Day 10 - Oil-price engine

Build: Model direct fuel impact plus configurable indirect pass-through to transport, food and utilities. Require units and distinguish crude-price assumptions from retail fuel prices.

Acceptance: Renter, commuter and low-driving profiles show explainable differences; sensitivity controls update results deterministically.

Git checkpoint: `feat(oil): add direct and indirect expense model`

Day 11 - Income-tax engine

Build: Implement a versioned India income-tax rules interface, selected tax regime/slabs for the supported tax year, deductions assumptions and salary take-home delta.

Acceptance: Golden tax cases match independently calculated fixtures; the UI displays tax year/regime and refuses unsupported years instead of guessing.

Git checkpoint: feat(tax): add versioned income-tax scenario engine

Day 12 - Job-loss and salary-change engine

Build: Calculate runway, emergency-fund coverage, cash-flow deficit, goal delays and salary-change impact. Keep employment probability outside the deterministic core unless explicitly assumed.

Acceptance: Results distinguish stock balances from monthly flows; zero-income and partial-income cases do not crash; warnings are supportive and non-alarmist.

Git checkpoint: feat(income): model job loss and salary changes

Day 13 - Unified simulation orchestration

Build: Add a registry that routes each supported scenario to its calculator, normalizes outputs and assembles impacts, graph data, assumptions, warnings and evidence references.

Acceptance: All five scenarios return the same stable result envelope; API integration tests cover success and validation errors.

Git checkpoint: feat(engine): unify scenario orchestration

Day 14 - Results experience

Build: Create the results page with annual/monthly net effect, affected categories, before/after values, confidence, key assumptions and an expandable calculation breakdown.

Acceptance: Users can trace every headline number to component impacts; negative/positive/neutral effects are not communicated by color alone.

Git checkpoint: feat(results): build transparent impact dashboard

Week 2 gate: All five deterministic engines run through a consistent API and produce inspectable results.

Week 3 - AI boundary, evidence and comparison

Day 15 - Natural-language scenario parser

Build: Add an LLM adapter that converts a question into constrained scenario JSON. Validate against the canonical schema, add deterministic fallback forms and require user confirmation of extracted numbers.

Acceptance: A prompt corpus maps common phrasings to the correct scenario; malformed or unsupported requests fail safely; no free-form model value reaches the engine unchecked.

Git checkpoint: feat(ai): parse questions into validated scenarios

Day 16 - Grounded explanation generator

Build: Generate explanations only from result JSON, assumptions and retrieved evidence metadata. Add prompt-injection resistance, numeric consistency checks and a non-LLM template fallback.

Acceptance: The explanation cannot introduce an unseen monetary value; snapshot tests verify required caveats, source labels and uncertainty language.

Git checkpoint: feat(ai): add grounded result explanations

Day 17 - Evidence registry and source UX

Build: Create versioned source records for RBI, MOSPI, Income Tax Department and other primary datasets used by the five models. Surface publisher, title, URL, publication date, accessed date and applicable assumption.

Acceptance: Every causal edge and model parameter has an evidence or explicit-assumption label; broken/missing references fail a validation job.

Git checkpoint: `feat(evidence): add source-backed model registry`

Day 18 - Scenario comparison

Build: Let users compare baseline and up to three variants, such as unchanged, -0.5 percentage points and -1.0 percentage point repo-rate paths. Add aligned tables and charts.

Acceptance: Variants reuse the same profile and formula versions; difference calculations reconcile with individual results; share state is serializable.

Git checkpoint: `feat(compare): compare scenario variants`

Day 19 - Winners-and-losers personas

Build: Add transparent representative personas: student/renter, salaried homeowner, small-business owner and retiree/saver. Run the same scenario contract for each; allow side-by-side comparison.

Acceptance: Persona inputs are visible and editable; group results never imply population-level causal proof; comparison works for all five scenarios where applicable.

Git checkpoint: `feat(personas): add winners and losers view`

Day 20 - Saved scenarios and history

Build: Persist scenario inputs, engine version, result snapshot and timestamps. Add history, duplicate, rename and archive actions without storing raw prompts unnecessarily.

Acceptance: Reloaded results remain reproducible; users can only access their own records; changed engine versions are visibly identified.

Git checkpoint: `feat(history): persist reproducible simulations`

Day 21 - Privacy and security review

Build: Threat-model auth, profile data, share links, LLM prompts and API boundaries. Add rate limits, secure headers, redacted logging, input limits and data deletion/export flows.

Acceptance: Row-level security tests pass; secrets scan is clean; shared artifacts contain no private profile fields by default; abuse cases are documented.

Git checkpoint: `fix(security): harden personal-data boundaries`

Week 3 gate: Natural-language input is constrained, evidence is visible, comparisons work and personal data is protected.

Week 4 - Uncertainty, reporting, quality and launch

Day 22 - Monte Carlo engine

Build: Add seeded Monte Carlo simulations with explicit distributions and correlations for a small approved set of uncertain parameters. Return percentiles, probability of loss and sample metadata.

Acceptance: Same seed reproduces results; tests check distribution sanity and bounds; UI labels simulations as assumption-driven, not forecasts.

Git checkpoint: `feat(simulation): add reproducible Monte Carlo engine`

Day 23 - Uncertainty visualization

Build: Add percentile bands, histogram/density view, sensitivity ranking and plain-language uncertainty summary. Provide accessible tabular alternatives.

Acceptance: Charts show units, baseline and sample count; screen-reader users can access equivalent values; empty/degenerate distributions render safely.

Git checkpoint: `feat(charts): visualize uncertainty and sensitivity`

Day 24 - Share links and PDF reports

Build: Add explicit redaction controls, expiring/revocable share tokens and a print-ready report containing profile summary, scenario, results, graph, distributions, assumptions, evidence and disclaimer.

Acceptance: Shared links work without exposing account access; revoked/expired links fail; exported PDF has no clipped content and matches the saved result snapshot.

Git checkpoint: `feat(reporting): add private sharing and PDF export`

Day 25 - End-to-end test suite

Build: Automate critical journeys for authentication, profile editing, each scenario, comparison, saved history and sharing. Add API contract tests and seeded fixtures.

Acceptance: Critical tests pass locally and in CI; failures retain useful traces/screenshots; flaky tests are fixed rather than retried indefinitely.

Git checkpoint: `test(e2e): cover critical user journeys`

Day 26 - Performance and resilience

Build: Profile page loads and simulations; add caching, database indexes, request timeouts, cancellation, background execution where needed and graceful degradation if the LLM is unavailable.

Acceptance: Core deterministic scenarios remain usable during LLM failure; agreed performance budgets pass on representative data; error states offer recovery.

Git checkpoint: `perf(app): improve simulation responsiveness`

Day 27 - Accessibility and responsive polish

Build: Complete keyboard navigation, focus management, labels, contrast, reduced-motion behavior, chart alternatives and responsive fixes across the full product.

Acceptance: Automated checks have no critical violations; core flow is manually usable by keyboard at mobile and desktop widths.

Git checkpoint: `fix(a11y): complete accessible responsive flows`

Day 28 - Deployment and observability

Build: Provision production/staging environments, migrations, domains, analytics with privacy controls, error reporting, health checks, backups and rollback notes.

Acceptance: Staging smoke tests pass against production-like services; alerts and health endpoints work; deployment does not depend on local secrets.

Git checkpoint: `chore(deploy): provision staging and observability`

Day 29 - Release candidate and user acceptance

Build: Run a structured acceptance pass with the five scenario families and four personas. Fix release-blocking defects, freeze schema/formula versions and prepare demo profiles plus walkthrough content.

Acceptance: No open severity-1 or severity-2 defects; calculation fixtures and sources are reviewed; release checklist is signed off.

Git checkpoint: `fix(release): stabilize RippleLab MVP candidate`

Day 30 - Production launch and handover

Build: Deploy production, run smoke tests, tag `v0.1.0`, publish architecture/formula/source documentation, record known limitations and create the post-MVP backlog.

Acceptance: Production critical journey works; rollback is documented; final PDF report links evidence and test results; GitHub release/tag is reproducible.

Git checkpoint: `chore(release): launch RippleLab v0.1.0`

Week 4 gate: A secure, tested and deployed MVP is demonstrable from question to source-backed personal impact report.

7. Daily Operating Rhythm

Each build day follows the same loop:

1. **Start:** Pull the current branch, inspect open issues and restate the day's acceptance criteria.
2. **Design:** Confirm contracts, migrations and UI states before implementation.
3. **Build:** Implement the smallest vertical increment that satisfies the day's objective.
4. **Verify:** Run targeted tests first, then lint, type-check, broader tests and production build as applicable.
5. **Review:** Inspect the diff for secrets, unintended files, numerical correctness, accessibility and scope creep.
6. **Document:** Update architecture/formula/source docs and generate the daily PDF report.
7. **Publish:** Commit and push only the verified changes; record commit SHA, branch and CI status in the report.

Recommended branch approach for a solo one-month build: use short-lived `codex/day-XX-topic` branches, push daily, and merge through a pull request after checks pass. Keep `main` deployable.

8. Daily PDF Report Contract

Every daily report will be named `RippleLab_Day_XX_Progress_Report.pdf` and include:

- Day, date, branch, commit SHA and build status.
- Objective and acceptance criteria.
- Completed work in user-facing language.
- Technical implementation and important files changed.
- Calculation formulas or data assumptions added/changed.
- Tests and verification with actual command results.
- Screenshots or diagrams when they materially clarify the result.
- Security, privacy, accessibility and data-quality notes.
- Decisions made, known limitations, risks and blockers.
- Exact scope proposed for the next day.

Reports must say "not completed" when a criterion is not met. They must never claim a push, deployment or passing test without verifiable evidence.

9. Quality Gates

Calculation correctness

- Monetary calculations use decimal-safe arithmetic and documented rounding.
- Each engine has golden fixtures independently derived from documented examples.
- Units are explicit: percentage versus percentage points, monthly versus annual, rupees versus percentages.
- Formula, source and result schemas are versioned.

AI safety and reliability

- Structured output is schema validated.
- User-visible numbers must originate from the deterministic engine.
- Unsupported scenarios are explicit.
- Model prompts exclude unnecessary personal data and resist instruction injection from retrieved text.

Engineering quality

- CI runs formatting/lint, type-check, unit/integration tests, build and secret scanning.
- Database migrations are reversible where practical and tested on staging.
- Critical journeys have end-to-end coverage.
- Dependencies and source snapshots are pinned/versioned.

Product quality

- Every result explains what changed, why, by how much, under what assumptions and with what confidence.
- The UI distinguishes observed data, user input, assumptions and simulated outputs.
- Core flows meet accessibility and mobile-use requirements.

10. Major Risks and Planned Responses

Risk	Early warning	Response
Scope exceeds one month	Daily work spills across two days	Preserve five scenarios; cut decorative features before correctness, evidence or tests
LLM produces invalid or invented values	Parser/explanation consistency tests fail	Strict schemas, deterministic fallback forms and numeric allow-list checks
Economic claims appear more certain than evidence	Single-number outputs dominate	Show ranges, sensitivities, confidence and editable assumptions
Tax/policy data becomes stale	Missing effective date or tax year	Version rules and sources; display applicability; reject unsupported periods
Sensitive data leaks through logs/shares	Raw profile or prompt appears in telemetry	Redaction, minimized prompts, private defaults and automated checks
Frontend and engine contracts drift	Type or integration tests fail	Canonical schema plus generated clients and contract CI
Monte Carlo becomes decorative	No documented distributions or sensitivity	Limit uncertain parameters; disclose distributions; seed and test runs
Deployment consumes the final week	Staging first appears after Day 28	Establish CI early and a basic staging path before polish if schedule slips

11. Scope-Cut Order if Time Slips

Cut in this order while protecting the core promise:

1. Advanced chart animation and decorative polish.
2. More than four comparison personas.
3. Revocable public share links (retain local/private PDF export).
4. LLM-generated prose (retain deterministic templated explanations).
5. Broad Monte Carlo support (retain sensitivity ranges for all scenarios and Monte Carlo for two strongest scenarios).

Do not cut calculation tests, evidence traceability, privacy controls, scenario validation or the five deterministic scenario families.

12. Inputs Needed Before the First Push

- GitHub repository URL or permission to create/connect one.
- Preferred application name/domain if different from RippleLab.
- Supabase project credentials, added through local/hosting secrets rather than chat or commits.
- Choice of LLM provider and API credentials when Day 15 begins.
- Deployment accounts for Vercel and Render/Railway before Day 28.
- Confirmation of the supported India tax year before implementing Day 11.

13. Day 30 Success Checklist

- Five supported scenarios work for a saved personal profile.
- Numeric outputs are deterministic, tested and traceable.

- Natural-language parsing is validated and has a form fallback.
- Causal graphs expose sources, assumptions and confidence.
- Scenario and persona comparisons work.
- Monte Carlo output is reproducible and responsibly explained.
- PDF export and privacy-safe sharing work.
- CI, staging, production monitoring and rollback instructions exist.
- Production smoke tests pass and release v0.1.0 is tagged.
- All 30 daily reports and the final handover report are available.